

UNIVERSITÉ DU QUÉBEC À CHICOUTIMI

Département d'informatique et de mathématique

---

# Optimisation de la production d'une centrale hydroélectrique

Modélisation par régression et programmation dynamique

---

**Cours :** 8INF926 - Atelier en optimisation avancée (Hiver 2026)

**Auteurs :** Francis LAVOIE  
Armand BEHAREL

**Professeure :** Sara Séguin

**Date :** 16 mars 2026

Remis au Département d'informatique et de mathématique

# Table des matières

|           |                                                                |           |
|-----------|----------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Introduction</b>                                            | <b>3</b>  |
| <b>I</b>  | <b>Partie 1 - Modélisation des fonctions de production</b>     | <b>5</b>  |
| <b>2</b>  | <b>Méthodes de modélisation de fonctions</b>                   | <b>6</b>  |
| 2.1       | Ajustement de courbes ( <i>curve fitting</i> ) . . . . .       | 6         |
| 2.2       | Moindres carrés linéaires . . . . .                            | 6         |
| 2.3       | Interpolation polynomiale . . . . .                            | 7         |
| 2.4       | Splines . . . . .                                              | 7         |
| 2.5       | Plans tangents et modélisation linéaire par morceaux . . . . . | 8         |
| 2.6       | Méthode retenue . . . . .                                      | 8         |
| <b>3</b>  | <b>Méthodologie retenue - Partie 1</b>                         | <b>9</b>  |
| 3.1       | Description des données . . . . .                              | 9         |
| 3.2       | Pré-traitement des données . . . . .                           | 9         |
| 3.3       | Modélisation de l'élévation avale . . . . .                    | 10        |
| 3.4       | Modélisation des fonctions de production . . . . .             | 10        |
| 3.5       | Choix du degré et validation . . . . .                         | 10        |
| <b>4</b>  | <b>Résultats - Partie 1</b>                                    | <b>12</b> |
| 4.1       | Modèle de l'élévation avale . . . . .                          | 12        |
| 4.2       | Coefficients des modèles polynomiaux - Turbines . . . . .      | 13        |
| 4.3       | Visualisation des modèles - Turbines . . . . .                 | 14        |
| 4.4       | Analyse des résultats de modélisation . . . . .                | 16        |
| <b>II</b> | <b>Optimisation par programmation dynamique</b>                | <b>17</b> |
| <b>5</b>  | <b>Programmation dynamique</b>                                 | <b>18</b> |

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| 5.1      | Principe général . . . . .                            | 18        |
| 5.2      | Les prérequis de la programmation dynamique . . . . . | 18        |
| 5.3      | Approche top-down et bottom-up . . . . .              | 19        |
| <b>6</b> | <b>Méthodologie, Partie 2</b>                         | <b>21</b> |
| 6.1      | Modélisation mathématique . . . . .                   | 21        |
| 6.1.1    | Variables et paramètres . . . . .                     | 21        |
| 6.1.2    | Problème d'optimisation . . . . .                     | 21        |
| 6.1.3    | Formulation récursive . . . . .                       | 22        |
| 6.2      | Implémentation en Python . . . . .                    | 22        |
| 6.2.1    | Choix du langage et librairies utilisées . . . . .    | 22        |
| 6.2.2    | Structure du code . . . . .                           | 22        |
| <b>7</b> | <b>Résultats, Partie 2</b>                            | <b>24</b> |
| 7.1      | Comparaison avec les données de référence . . . . .   | 24        |
| 7.2      | Temps de résolution . . . . .                         | 25        |
| 7.3      | Effet de la modélisation sur la précision . . . . .   | 25        |
| <b>8</b> | <b>Discussion</b>                                     | <b>27</b> |
| <b>9</b> | <b>Conclusion et travaux futurs</b>                   | <b>28</b> |

# Chapitre 1

## Introduction

La production hydroélectrique constitue l'une des formes d'énergie renouvelable les plus répandues au monde, et particulièrement au Québec où elle représente la principale source d'électricité. Une centrale hydroélectrique transforme l'énergie potentielle de l'eau en énergie électrique grâce à des turbines couplées à des alternateurs. La puissance produite par chaque turbine dépend de deux paramètres fondamentaux : la hauteur de chute nette (différence entre les niveaux amont et aval, corrigée des pertes hydrauliques) et le débit turbiné.

Dans un contexte de gestion optimale des ressources, l'opérateur d'une centrale doit résoudre quotidiennement un problème d'allocation : comment répartir un débit total disponible entre les turbines de la centrale afin de maximiser la puissance électrique totale produite ? Ce problème, bien que conceptuellement simple, se révèle non trivial car les fonctions de production des turbines sont non linéaires et couplées par l'élévation aval, qui dépend du débit total.

Ce projet porte sur la centrale de la rivière des Outaouais et s'articule en deux parties complémentaires.

La **première partie** vise à établir des représentations analytiques des fonctions de production à partir de données réelles de la centrale, collectées entre 2013 et 2017 à une fréquence de deux minutes. Ces données incluent les débits individuels et totaux, les puissances produites par chaque turbine ainsi que les niveaux amont et aval. On cherche à modéliser, pour chaque turbine, la puissance en fonction de la hauteur de chute nette et du débit, ainsi que l'élévation aval en fonction du débit total. L'outil retenu est la boîte à outils *Curve Fitting Toolbox* de MATLAB.

La **deuxième partie** exploite ces fonctions analytiques comme fonctions objectif dans un algorithme de programmation dynamique (PD), implémenté en Python. L'objectif est de maximiser la puissance totale pour un débit total et un niveau amont donnés, en

déterminant la répartition optimale du débit entre les cinq turbines. L'implémentation repose sur une approche par récurrence en avant discrétisée, avec reconstruction de la solution optimale par retour arrière.

Le présent rapport décrit la démarche méthodologique retenue, les résultats obtenus et une analyse critique des performances de l'optimisation.

# Première partie

## Partie 1 - Modélisation des fonctions de production

# Chapitre 2

## Méthodes de modélisation de fonctions

### 2.1 Ajustement de courbes (*curve fitting*)

L'ajustement de courbes est une technique permettant de trouver une fonction mathématique qui représente au mieux un ensemble de données expérimentales. Contrairement à l'interpolation, qui force le modèle à passer exactement par chaque point, l'ajustement de courbes cherche une fonction qui minimise globalement l'écart aux données, ce qui le rend plus robuste au bruit de mesure. L'outil principal utilisé dans le cadre de ce projet est la *Curve Fitting Toolbox* de MATLAB, qui offre une interface graphique permettant de visualiser les données, d'ajuster différents types de modèles et d'évaluer leur qualité à l'aide de métriques telles que le RMSE (*Root Mean Square Error*) et le coefficient de détermination  $R^2$ .

### 2.2 Moindres carrés linéaires

La méthode des moindres carrés linéaires (*Linear Least Squares*) est la technique d'ajustement la plus répandue. Elle consiste à minimiser la somme des carrés des résidus entre les valeurs observées et les valeurs prédites par le modèle :

$$\min_{\boldsymbol{\beta}} \sum_{i=1}^n (y_i - f(\mathbf{x}_i, \boldsymbol{\beta}))^2 \quad (2.1)$$

Lorsque le modèle est linéaire par rapport aux paramètres  $\boldsymbol{\beta}$  (ce qui est le cas des polynômes), ce problème admet une solution analytique fermée via l'équation normale :

$$\boldsymbol{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \quad (2.2)$$

où  $\mathbf{X}$  est la matrice de conception contenant les termes du polynôme évalués aux points de données. Cette approche est efficace, exacte et ne nécessite pas d'itérations, ce qui la rend particulièrement adaptée à l'ajustement de polynômes sur de grands jeux de données.

## 2.3 Interpolation polynomiale

L'interpolation polynomiale consiste à construire un polynôme de degré  $n - 1$  passant exactement par  $n$  points de données. Le polynôme de Lagrange en est la forme classique :

$$P(x) = \sum_{i=0}^{n-1} y_i \prod_{\substack{j=0 \\ j \neq i}}^{n-1} \frac{x - x_j}{x_i - x_j} \quad (2.3)$$

Bien que cette méthode garantisse le passage exact par tous les points, elle souffre du phénomène de Runge pour des degrés élevés : des oscillations importantes apparaissent aux extrémités du domaine. Pour des jeux de données de grande taille comme ceux de ce projet (plusieurs années à 2 minutes), l'interpolation polynomiale globale est donc inadaptée.

## 2.4 Splines

Les splines sont des fonctions polynomiales par morceaux raccordées en des points appelés nœuds, avec continuité des dérivées aux jonctions. Les splines cubiques naturelles, par exemple, sont des polynômes de degré 3 sur chaque intervalle, avec continuité de la fonction et de ses deux premières dérivées. Elles offrent une grande souplesse d'ajustement locale sans les oscillations des polynômes globaux de haut degré. En deux dimensions, les *thin-plate splines* permettent d'interpoler des surfaces de manière lisse. Cependant, les splines produisent des modèles moins compacts que les polynômes globaux et sont plus difficiles à intégrer directement dans un algorithme d'optimisation.



## 2.5 Plans tangents et modélisation linéaire par morceaux

Un plan tangent en un point  $(h_0, q_0)$  d'une surface  $P(h, q)$  est l'approximation linéaire locale de cette surface :

$$P(h, q) \approx P(h_0, q_0) + \left. \frac{\partial P}{\partial h} \right|_{(h_0, q_0)} (h - h_0) + \left. \frac{\partial P}{\partial q} \right|_{(h_0, q_0)} (q - q_0) \quad (2.4)$$

Cette approche est valide uniquement dans un voisinage restreint du point de linéarisation. La modélisation linéaire par morceaux généralise ce concept en divisant le domaine en régions et en ajustant un modèle linéaire sur chacune. Elle permet une meilleure représentation globale tout en conservant la simplicité analytique, au prix d'une gestion de la discontinuité aux frontières entre régions.

## 2.6 Méthode retenue

Pour ce projet, la régression polynomiale multivariée par moindres carrés linéaires (*Linear Least Squares*) a été retenue, en utilisant la forme `poly33` de la *Curve Fitting Toolbox* de MATLAB. Le choix de cette méthode d'ajustement s'est fait naturellement puisqu'il s'agit de la méthode par défaut de MATLAB pour l'ajustement de polynômes. La forme analytique compacte obtenue s'intègre par ailleurs directement dans l'algorithme de programmation dynamique comme fonction objectif évaluable en temps constant.

# Chapitre 3

## Méthodologie retenue - Partie 1

### 3.1 Description des données

Les données brutes sont contenues dans un fichier CSV couvrant la période 2013-2017, à une fréquence d'acquisition de 2 minutes. Chaque ligne représente un instantané des mesures aux capteurs de la centrale. Le tableau 3.1 décrit les colonnes disponibles.

TABLE 3.1 – Description des colonnes du fichier de données

| Colonne            | Description                                           |
|--------------------|-------------------------------------------------------|
| Elav               | Élévation avale (m)                                   |
| Qtot               | Débit total turbiné à la centrale (m <sup>3</sup> /s) |
| Qvan               | Débit total déversé à la centrale (m <sup>3</sup> /s) |
| Niv_Amont          | Élévation amont (m)                                   |
| Q1, Q2, Q3, Q4, Q5 | Débit turbiné par chaque turbine (m <sup>3</sup> /s)  |
| P1, P2, P3, P4, P5 | Puissance produite par chaque turbine (MW)            |

### 3.2 Pré-traitement des données

Les données brutes ont été examinées et prétraitées dans Microsoft Excel. La qualité des données s'est révélée très satisfaisante : aucune valeur aberrante n'a nécessité de suppression, et les séries temporelles ne présentaient pas de lacune significative.

Les colonnes suivantes ont été créées pour chaque ligne :

1. **Hauteur brute** :  $H = \text{Niv\_amont} - \text{Elav}$

2. **Hauteur nette par turbine** : pour  $i \in \{1, \dots, 5\}$ ,

$$H_i^{\text{net}} = H - 0,5 \times 10^{-5} \times Q_i^2 \quad (3.1)$$

3. **Puissance totale mesurée** :  $P_{\text{tot}} = \sum_{i=1}^5 P_i$ , servant de variable de contrôle.

Les observations pour lesquelles  $Q_i = 0$  ont été exclues de l'ajustement de la turbine  $i$ , car elles correspondent à une turbine hors service (pas de relation physique entre débit et puissance dans ce cas).

### 3.3 Modélisation de l'élévation avale

L'élévation avale Elav a été modélisée comme un polynôme de degré 2 du débit total  $Q_{\text{tot}}$  :

$$\text{Elav}(Q_{\text{tot}}) = -1,4527 \times 10^{-6} Q_{\text{tot}}^2 + 0,007 Q_{\text{tot}} + 99,9812 \quad (3.2)$$

Le degré 2 a été retenu plutôt que le degré 1 : bien que ce dernier produise une RMSE acceptable (0,1624 m), il ne capte pas correctement le comportement en fin de courbe où les données sont peu denses. Le degré 2 corrige cet écart avec une RMSE de 0,1251 m et un  $R^2 = 0,9763$ .

### 3.4 Modélisation des fonctions de production

Pour chaque turbine  $i$ , la puissance a été modélisée par un polynôme de degré 3 en deux variables ( $H_i^{\text{net}}$  et  $Q_i$ ), selon la forme `poly33` de MATLAB :

$$P_i(h, q) = \sum_{\substack{j, k \geq 0 \\ j+k \leq 3}} p_{jk}^{(i)} h^j q^k \quad (3.3)$$

soit 10 coefficients par turbine. L'ajustement a été réalisé avec la *Curve Fitting Toolbox* de MATLAB, en utilisant la méthode des moindres carrés linéaires (*Linear Least Squares*).

### 3.5 Choix du degré et validation

Le degré 3 a été retenu après comparaison systématique des degrés 1, 2 et 3. Le degré 1 sous-ajuste clairement ( $\text{RMSE} > 2 \text{ MW}$ ). Le degré 2 donne des RMSE acceptables mais systématiquement supérieures à 0,25 MW pour certaines turbines. Le degré 3 permet d'atteindre  $\text{RMSE} < 0,25 \text{ MW}$  pour toutes les turbines, avec  $R^2 > 0,997$ . Le degré 4 a été

testé et montrait des signes d'overfitting : puissances négatives pour certaines combinaisons  $(h, q)$  physiquement valides, et dégradation du RMSE sur un sous-ensemble de validation.

# Chapitre 4

## Résultats - Partie 1

### 4.1 Modèle de l'élévation avale

Le tableau 4.1 présente les coefficients, intervalles de confiance à 95% et métriques de qualité du modèle `poly2` obtenu pour l'élévation avale.

TABLE 4.1 – Coefficients du modèle `poly2` - Élévation avale en fonction de  $Q_{\text{tot}}$

| Coefficient  | Valeur                   | Borne inf. (95%)         | Borne sup. (95%)         |
|--------------|--------------------------|--------------------------|--------------------------|
| $p_1$        | $-1,4527 \times 10^{-6}$ | $-1,4691 \times 10^{-6}$ | $-1,4362 \times 10^{-6}$ |
| $p_2$        | 0,0070                   | 0,0070                   | 0,0070                   |
| $p_3$        | 99,9812                  | 99,9702                  | 99,9922                  |
| SSE          |                          | 685,6107                 |                          |
| $R^2$        |                          | 0,9763                   |                          |
| $R^2$ ajusté |                          | 0,9763                   |                          |
| RMSE         |                          | 0,1251 m                 |                          |

La figure 4.1 présente le nuage de points des données réelles superposé à la courbe ajustée.

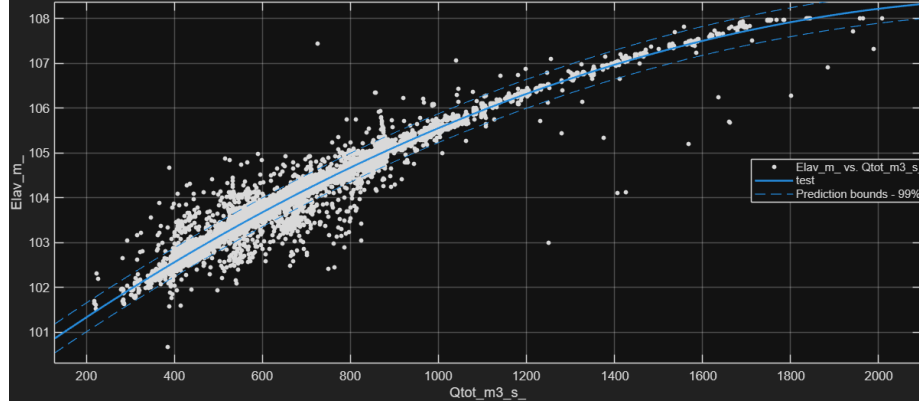


FIGURE 4.1 – Élévation aval en fonction du débit total - données réelles et modèle poly2

## 4.2 Coefficients des modèles polynomiaux - Turbines

Le tableau 4.2 présente les coefficients des modèles poly33 obtenus pour chacune des cinq turbines.

TABLE 4.2 – Coefficients des modèles poly33 pour chaque turbine

| Coeff.   | T1                      | T2                      | T3                      | T4                      | T5                      |
|----------|-------------------------|-------------------------|-------------------------|-------------------------|-------------------------|
| $p_{00}$ | -1405,1                 | -836,457                | -836,908                | -2614,1                 | -662,610                |
| $p_{10}$ | 114,06                  | 71,812                  | 66,968                  | 192,253                 | 50,320                  |
| $p_{01}$ | 1,6455                  | -0,934                  | 0,864                   | 7,888                   | 0,648                   |
| $p_{20}$ | -3,0733                 | -2,054                  | -1,774                  | -4,586                  | -1,251                  |
| $p_{11}$ | -0,1026                 | 0,0352                  | -0,051                  | -0,450                  | -0,043                  |
| $p_{02}$ | 0,0045                  | 0,0072                  | 0,003                   | 0,0024                  | 0,004                   |
| $p_{30}$ | 0,0275                  | 0,0196                  | 0,016                   | 0,0351                  | 0,010                   |
| $p_{21}$ | 0,0016                  | $-2,944 \times 10^{-4}$ | $8 \times 10^{-4}$      | 0,0064                  | $8 \times 10^{-4}$      |
| $p_{12}$ | $-4,754 \times 10^{-6}$ | $-5,205 \times 10^{-5}$ | $1,969 \times 10^{-6}$  | $8,982 \times 10^{-5}$  | $-3,147 \times 10^{-5}$ |
| $p_{03}$ | $-1,591 \times 10^{-5}$ | $-1,939 \times 10^{-5}$ | $-1,155 \times 10^{-5}$ | $-1,956 \times 10^{-5}$ | $-1,180 \times 10^{-5}$ |
| $R^2$    | 0,9998                  | 0,9998                  | 0,9999                  | 0,9972                  | 0,9998                  |
| RMSE     | 0,2289                  | 0,2339                  | 0,1800                  | 0,2431                  | 0,1968                  |

Tous les modèles satisfont le critère  $RMSE < 0,25$  MW et affichent un  $R^2$  supérieur à 0,997, ce qui atteste d'une très bonne qualité d'ajustement. La turbine T4 présente le  $R^2$  légèrement plus faible (0,997) et le RMSE le plus élevé (0,243), ce qui est cohérent avec une plage de fonctionnement plus étendue et une plus grande sensibilité à la hauteur de

chute.

### 4.3 Visualisation des modèles - Turbines

Les figures 4.2 à 4.6 présentent, pour chaque turbine, le nuage de points des données réelles superposé à la surface poly33 ajustée.

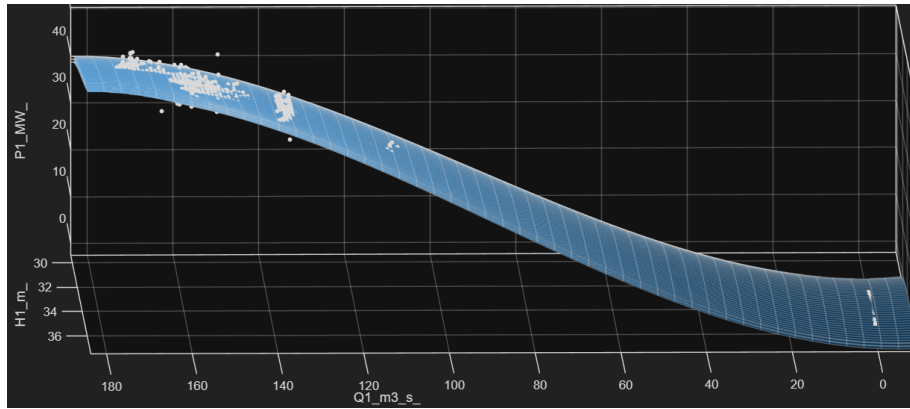


FIGURE 4.2 – Puissance produite en fonction de la hauteur nette et du débit - Turbine 1

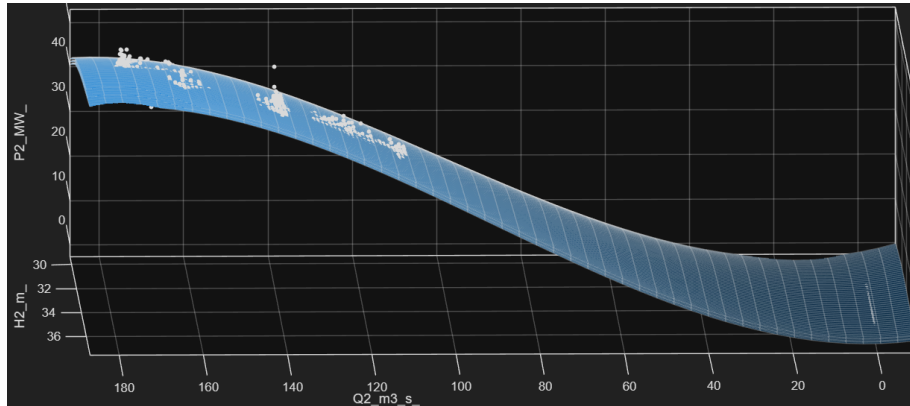


FIGURE 4.3 – Puissance produite en fonction de la hauteur nette et du débit - Turbine 2

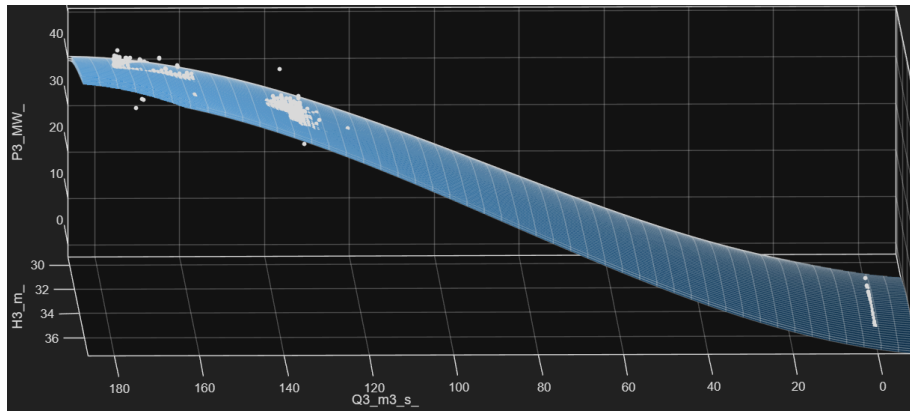


FIGURE 4.4 – Puissance produite en fonction de la hauteur nette et du débit - Turbine 3

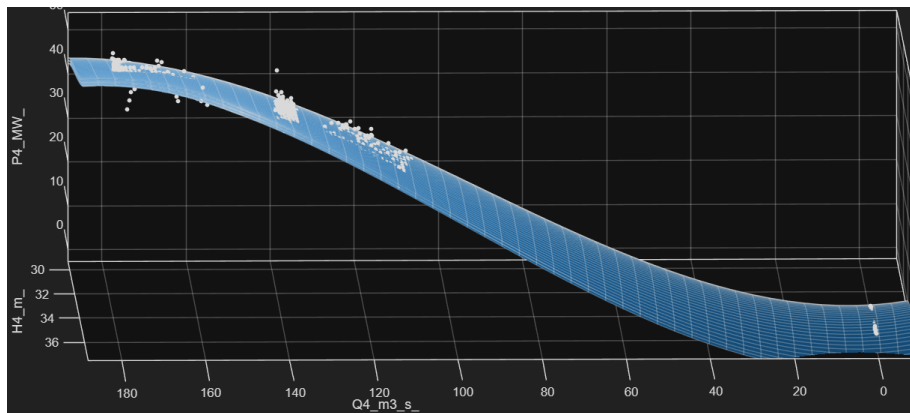


FIGURE 4.5 – Puissance produite en fonction de la hauteur nette et du débit - Turbine 4

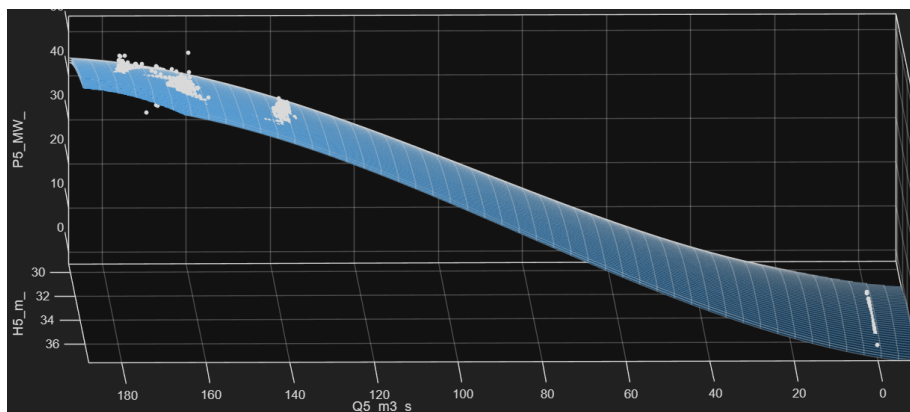


FIGURE 4.6 – Puissance produite en fonction de la hauteur nette et du débit - Turbine 5



## 4.4 Analyse des résultats de modélisation

Les modèles polynomiaux obtenus reproduisent fidèlement les « collines de rendement » caractéristiques des turbines. Pour chaque turbine, la puissance croît avec la hauteur nette et avec le débit jusqu'à un maximum, puis décroît légèrement aux débits très élevés, ce qui est physiquement cohérent.

Le coefficient  $p_{10}$  (terme linéaire en  $h$ ) est le plus dominant pour toutes les turbines, ce qui reflète la forte dépendance linéaire de la puissance à la hauteur de chute. Les termes quadratiques et cubiques en  $h$  et  $q$  permettent de capturer les non-linéarités et les effets croisés.

Deuxième partie

Optimisation par programmation  
dynamique

# Chapitre 5

## Programmation dynamique

### 5.1 Principe général

La programmation dynamique (PD) est une méthode d'optimisation introduite par Richard Bellman dans les années 1950 (bellman1957-1). Celle-ci vise à optimiser des calculs complexes tout en limitant le temps de calcul nécessaire à l'obtention de la solution optimale. En effet, les méthodes naïves telles que le calcul de toutes les combinaisons possibles sont souvent impossibles à réaliser en pratique, entre autres puisque le temps de calcul serait trop élevé. Il a donc été nécessaire de développer des approches permettant de trouver une solution optimale tout en réduisant le nombre de calculs à effectuer. Pour ce faire, la programmation dynamique est une technique algorithmique qui consiste à découper un problème complexe en plusieurs sous-problèmes. L'idée principale derrière cela est de calculer la solution optimale de chacun des sous-problèmes. On garde ensuite en mémoire la solution optimale de ceux-ci afin d'éviter les calculs redondants. On évite donc de calculer plusieurs fois la même chose, ce qui réduit le temps de calcul et permet de limiter l'effet exponentiel de l'ajout de dimensionnalités. Évidemment, la programmation dynamique est tout de même sensible à l'augmentation des dimensionnalités, mais beaucoup moins qu'un algorithme qui testerait toutes les combinaisons possibles, par exemple. La dernière étape de la programmation dynamique consiste à reconstruire la solution finale à partir des solutions optimales de chacun des sous-problèmes.

### 5.2 Les prérequis de la programmation dynamique

Pour qu'un problème puisse être résolu efficacement et de façon optimale grâce à la programmation dynamique, celui-ci doit généralement respecter les deux propriétés suivantes : la sous-structure optimale et les sous-problèmes chevauchants. La sous-structure

optimale repose sur le principe *principe d'optimalité* : toute sous-politique d'une politique optimale est elle-même optimale. Autrement dit, on peut résoudre un problème global grâce à la combinaison des meilleures solutions de ses sous-problèmes. Un exemple de cette propriété apparaît dans les problèmes du plus court chemin dans un graphe. En effet, si le plus court chemin entre les sommets A et C passe par le sommet B, alors les chemins entre A et B et B et C doivent eux aussi être des plus courts chemins.

La deuxième propriété importante associée à la programmation dynamique est la présence de sous-problèmes chevauchants. Cela signifie que les mêmes sous-problèmes apparaissent plusieurs fois lors de la résolution du problème initial. Dans une approche naïve, ces sous-problèmes seraient calculés plusieurs fois. C'est dans ces cas que la programmation dynamique devient particulièrement efficace, puisque l'on calcule chaque sous-problème une seule fois, on stocke l'information dans une structure de données telle qu'un tableau et on construit la solution optimale à partir de ces sous-problèmes optimaux. Cela a comme effet d'éliminer les calculs redondants, réduisant par le fait même le temps de calcul nécessaire et la sensibilité à l'augmentation des dimensionnalités.

### 5.3 Approche top-down et bottom-up

La programmation dynamique peut être mise en œuvre selon deux approches principales : l'approche *top-down* et l'approche *bottom-up*. Bien que ces deux méthodes reposent sur les mêmes principes fondamentaux, elles diffèrent dans la manière dont les sous-problèmes sont explorés et résolus.

L'approche *top-down*, consiste à résoudre le problème initial de manière récursive en le décomposant progressivement en sous-problèmes plus petits. Lorsqu'un sous-problème est résolu pour la première fois, sa solution est mémorisée dans une structure de données afin d'éviter de recalculer ce même sous-problème ultérieurement. Lorsqu'un sous-problème déjà résolu apparaît de nouveau, l'algorithme récupère directement la solution stockée plutôt que de la recalculer.

De son côté, l'approche *bottom-up*, adopte une stratégie différente. Au lieu de commencer par le problème global, l'algorithme résout d'abord les sous-problèmes les plus simples, puis combine progressivement leurs solutions afin de résoudre des sous-problèmes de plus en plus grands. Concrètement, les solutions des sous-problèmes sont généralement stockées dans un tableau qui est rempli de manière itérative. Les premières entrées du tableau correspondent aux cas de base du problème, tandis que les entrées suivantes sont calculées à partir des valeurs déjà déterminées. Dans la pratique, le choix entre ces deux approches dépend généralement de la structure du problème étudié et des contraintes de

performance associées à son implémentation.

# Chapitre 6

## Méthodologie, Partie 2

### 6.1 Modélisation mathématique

#### 6.1.1 Variables et paramètres

- $N$  : nombre de turbines disponibles
- $Q_{\text{total}}$  : débit total à répartir ( $\text{m}^3/\text{s}$ )
- $Q_k$  : débit alloué à la turbine  $k$  (variable de décision)
- $Q_{\text{max}}$  : débit maximum par turbine ( $\text{m}^3/\text{s}$ )
- $\Delta Q$  : palier de discrétisation ( $\text{m}^3/\text{s}$ )
- $H_{k,\text{nette}}$  : hauteur nette de la turbine  $k$  (m)
- $P_k(H_{k,\text{nette}}, Q_k)$  : puissance de la turbine  $k$  (MW)

#### 6.1.2 Problème d'optimisation

$$\max_{Q_1, \dots, Q_N} \sum_{k=1}^N P_k(H_{k,\text{nette}}, Q_k) \quad (6.1)$$

sous les contraintes :

$$\sum_{k=1}^N Q_k = Q_{\text{total}} \quad (6.2)$$

$$0 \leq Q_k \leq Q_{\text{max}}, \quad \forall k = 1, \dots, N \quad (6.3)$$

### 6.1.3 Formulation récursive

On définit  $V_k(q)$  comme la puissance maximale atteignable par les  $k$  premières turbines pour un débit disponible  $q$  :

$$V_k(q) = \max_{0 \leq x \leq q, x \in \mathcal{Q}} [P_k(H_{k,\text{nette}}, x) + V_{k-1}(q - x)] \quad (6.4)$$

avec la condition initiale  $V_1(q) = P_1(H_{1,\text{nette}}, q)$ .

La solution optimale est  $V_N(Q_{\text{total}})$ , reconstruite par remontée des décisions mémorisées.

## 6.2 Implémentation en Python

### 6.2.1 Choix du langage et librairies utilisées

L'algorithme de programmation dynamique a été implémenté en Python. Ce choix a été motivé par l'accessibilité de ce langage en plus de notre connaissance de celui-ci. En effet, ce langage possède une grande communauté, beaucoup de documentation en plus d'offrir plusieurs librairies axées sur le domaine scientifique. Les principales librairies utilisées lors de ce projet sont les suivantes :

- *flask* : framework principal, permet de faire une api et une interface web en python (?).
- *pandas* : lecture des données dans le fichier xlsx fourni (?).
- *requests* : permet de faire des requêtes http en python (?).

Ces librairies ont permis de faire une interface web permettant de fournir le niveau amont et le débit total. Le programme nous affiche ensuite la répartition optimale entre les turbines et la puissance totale produite. Une page de base permet également de comparer les résultats obtenus grâce à l'algorithme à ceux du fichier xlsx fourni.

### 6.2.2 Structure du code

Le code Python est séparé en trois fichiers principaux. Tout d'abord, le fichier *interface.py* contient toutes les routes de l'API *Flask*. Ces routes permettent à l'utilisateur de naviguer entre les différentes pages. Cette partie sera améliorée lors de la troisième partie de ce projet. Ensuite, le fichier *turbine.py* contient la classe turbine permettant de calculer la puissance de chaque turbine en fonction du débit turbiné. Ce calcul est effectué grâce aux fonctions estimées qui ont été détaillées dans la première partie de ce rapport.

Finalement, le fichier *centrale.py* contient le code portant sur l'algorithme de programmation dynamique. En effet, la classe centrale est responsable, entre autres, d'initialiser les turbines, de résoudre chaque sous-problème et de construire la solution optimale grâce à l'approche *bottom-up*.



# Chapitre 7

## Résultats, Partie 2

### 7.1 Comparaison avec les données de référence

Tel que le montre le tableau 7.1, le modèle de programmation dynamique produit des résultats satisfaisants. En effet, sur 100 problèmes analysés, le modèle présente une erreur relative moyenne d'environ 5.11%, soit une différence moyenne d'environ 9.09 MW. De plus, le nombre de turbines activées pour chaque problème correspond exactement à celui fourni dans le fichier `xlsx`. Considérant que les fonctions de production de chacune des turbines ont été estimées, ces résultats peuvent être considérés comme très satisfaisants.

TABLE 7.1 – Comparaison entre la puissance réelle et estimée.

| Ligne    | $P_{rel}$ | $P_{estim}$ | $\Delta P$ | Turbines |
|----------|-----------|-------------|------------|----------|
| 0        | 179.17    | 190.07      | 10.90      | 4        |
| 1        | 179.50    | 190.02      | 10.52      | 4        |
| 2        | 179.13    | 190.02      | 10.89      | 4        |
| 3        | 179.51    | 190.02      | 10.51      | 4        |
| 4        | 178.30    | 188.73      | 10.42      | 4        |
| $\vdots$ | $\vdots$  | $\vdots$    | $\vdots$   | $\vdots$ |
| 95       | 169.64    | 180.28      | 10.64      | 4        |
| 96       | 169.25    | 180.23      | 10.97      | 4        |
| 97       | 167.79    | 178.71      | 10.92      | 4        |
| 98       | 168.55    | 178.76      | 10.21      | 4        |
| 99       | 166.61    | 178.71      | 12.10      | 4        |

## 7.2 Temps de résolution

Comme le montre le tableau 7.2, l’augmentation de la taille des paliers de discrétisation du débit permet de réduire significativement le temps de calcul tout en maintenant une erreur relative similaire. Bien que ce temps de calcul ne soit pas dépendant de la puissance de la machine qui l’exécute, on peut voir que plus les paliers sont grands, plus le temps de calcul nécessaire diminue. L’erreur moyenne relative reste similaire, bien qu’il faudrait tester sur un plus vaste échantillon afin de confirmer cette tendance.

TABLE 7.2 – Impact du nombre de paliers sur le temps de calcul moyen et l’erreur relative.

| Palier | Temps de calcul moyen (s) | Erreur moyenne (MW) |
|--------|---------------------------|---------------------|
| 2      | 0.54                      | 9.09                |
| 5      | 0.14                      | 9.15                |
| 10     | 0.05                      | 9.10                |
| 20     | 0.02                      | 8.38                |

## 7.3 Effet de la modélisation sur la précision

Lors de la phase de modélisation, certaines turbines ont d’abord été testées avec un polynôme de degré 2 en deux variables (`poly22`), avant d’adopter définitivement le degré 3 (`poly33`). Bien que le modèle `poly22` produise des métriques d’ajustement apparemment acceptables, son comportement s’est révélé problématique lors de son intégration dans l’algorithme de programmation dynamique.

En effet, les surfaces `poly22` se comportaient davantage comme des plans ou des surfaces quasi-linéaires que comme des collines de rendement. Ce comportement est insuffisant pour représenter fidèlement la réalité physique des turbines, dont la puissance atteint un maximum pour une combinaison optimale de hauteur nette et de débit, puis décroît de part et d’autre. En l’absence de ce pic bien défini, l’algorithme de programmation dynamique concentrait la totalité du débit disponible sur une seule turbine, cherchant à exploiter la croissance monotone de la surface modélisée plutôt qu’une répartition équilibrée entre les turbines. Ce comportement ne correspondait pas aux données réelles, où les turbines fonctionnent typiquement dans des plages de débit comparables.

Le passage au modèle `poly33`, grâce aux termes cubiques supplémentaires, a permis de capturer correctement la courbure des collines de rendement. L’algorithme produit dès lors une répartition physiquement cohérente du débit entre les turbines, confirmant que la

qualité de la modélisation des fonctions de production a un impact direct sur la validité de la solution optimale obtenue.

# Chapitre 8

## Discussion

Afin de compléter ce travail, certaines hypothèses ont été nécessaires. Premièrement, les données fournies par la centrale sont supposées exactes, c'est-à-dire que les mesures des capteurs sont considérées comme fiables et exemptes d'erreurs d'instrumentation ou de bruit de mesure significatif. Deuxièmement, les fonctions d'approximation poly33 obtenues sont supposées valides sur l'ensemble du domaine de fonctionnement des turbines, y compris dans les zones peu ou pas couvertes par les données.

Tel qu'expliqué précédemment, l'algorithme de programmation dynamique produit des résultats satisfaisants dans un délai raisonnable. Par le fait même, l'estimation des fonctions de production semble bien faite. En effet, la puissance moyenne obtenue par l'algorithme varie d'environ 5.11% par rapport aux données se trouvant dans le fichier *xlsx* fourni par la centrale hydroélectrique. De plus, avec un temps de résolution moyen en dessous d'une seconde, il est juste de dire que l'algorithme de programmation dynamique effectue l'optimisation pratiquement instantanément.

La mise en place de cet algorithme n'a pas été de tout repos. En effet, comme dans tout travail scolaire, il n'est pas toujours évident de saisir toutes les subtilités avant de se mettre au travail. Par exemple, il a fallu un certain temps avant que l'on comprenne à quoi allaient servir les fonctions estimées. Cela peut sembler un peu bizarre de ne pas avoir compris cette partie dès le début, mais tant qu'on a pas commencé concrètement un travail, il est difficile d'en voir la vue d'ensemble. L'algorithme de programmation dynamique n'a pas été simple à mettre en place également. Plusieurs facteurs ont contribué à cela, entre autres le fait qu'un membre de l'équipe ait dû manquer le cours complet portant sur la programmation dynamique. D'autre part, des difficultés ont été rencontrées lors de la création des structures de données servant à stocker les solutions optimales de chacun des sous-problèmes. Il a fallu étudier et comprendre les dimensions des tableaux à produire avant de se lancer. Notre enseignante s'est avérée très utile lors de cette étape.

# Chapitre 9

## Conclusion et travaux futurs

Pour conclure, ce laboratoire nous a permis de mieux comprendre plusieurs aspects de la programmation dynamique et de la résolution de problèmes d'optimisation. Tout d'abord, nous avons pu voir comment approximer des fonctions à l'aide de *Matlab*. Cet aspect du travail était particulièrement intéressant puisqu'il nous a montré concrètement comment il est possible de passer de données réelles à des fonctions pouvant être utilisées par la suite pour faire de la prédiction ou de l'optimisation. Par la suite, la deuxième partie du travail nous a permis d'approfondir notre compréhension de la programmation dynamique. Nous avons ainsi pu mieux comprendre le fonctionnement de cette approche d'optimisation ainsi que les différentes étapes nécessaires à son implémentation. Malgré les embûches rencontrées, nous sommes satisfaits de ce que nous avons réussi à accomplir et nous sommes impatients de poursuivre ce projet avec l'optimisation boîte noire qui sera ajoutée lors de la deuxième partie de ce projet ainsi qu'avec l'amélioration de l'interface graphique lors de la troisième partie.

# Bibliographie

bellman1957

- [1] R. Bellman, *Dynamic Programming*. Princeton University Press, 1957.